

Versión: MVP / hackathon

Stack principal: Next.js 16 (App Router), React 19, Supabase (Postgres + Auth + pgvector), Vercel AI SDK v6, Tailwind CSS 4 + shadcn/ui.

---

/ Landing MVP: CTAs a chat y dashboard, checklist de ingesta y env.  
/chat Chat asistido: MediChat (cliente).  
/dashboard Placeholder: tarjetas de medicación / síntomas / enlaces rápidos.  
/report Placeholder para PDF futuro hacia el médico.  
/login Magic link Supabase: formulario cliente + avisos de configuración.  
/auth/callback Route Handler: intercambio PKCE code → sesión (cookies en redirect).  
/auth/sign-out Cierre de sesión y redirect a /.

### 1.3 Componentes destacados

- components/features/chat/medichat.tsx: @ai-sdk/react (useChat), DefaultChatTransport → POST / api/chat. Input controlado, dictado Web Speech API (Chrome/Edge), manejo de errores de red/ servicio de voz, hidratación segura (clientReady antes de detectar SpeechRecognition).
- components/features/auth/login-form.tsx: signInWithOtp con createBrowserClient (@supabase/ssr), emailRedirectTo hacia /auth/callback?next=/dashboard.
- components/layout/site-header.tsx: Server Component: createClient() servidor, getUser(), navegación principal + Entrar / email + Salir.
- \*\*components/ui/\*: Primitivos shadcn (Button, Card, Input, ScrollArea, Badge, etc.).

### 1.4 Configuración front

- config/navigation.ts: MAIN\_NAV (Dashboard, Chat, Reporte); login se resuelve aparte en el header.
- middleware.ts: delega en lib/integrations/supabase/middleware.ts (updateSession) para refrescar cookies de sesión Supabase en cada request elegible.

---

## 2. Back-end (Next.js server)

### 2.1 API Routes

- POST /api/chat (app/api/chat/route.ts):
  - Valida credenciales: AI Gateway o Anthropic o OpenAI (chat); embeddings vía Gateway o OPENAI\_API\_KEY.
  - Modelo: resolveChatModel() / resolveEmbeddingModel() (lib/medicoach/ai/models.ts): Gateway usa IDs string tipo anthropic/...; directo: Anthropic o gpt-4o-mini (configurable OPENAI\_CHAT\_MODEL).
  - Flujo: convertToModelMessages + streamText (AI SDK), tools (createMediCoachTools), stopWhen(stepCountIs(8)), runIntentGraph (LangGraph/heurística) para enriquecer system prompt.
  - Persistencia: si hay usuario Supabase, onFinish → persistChatTurn (lib/medicoach/persistence/chat-session.ts) en chat\_sessions.

- GET /api/health (app/api/health/route.ts): comprobación básica de servicio.

## 2.2 Dominio MediCoach (lib/medicoach/)

- ai/: env.ts (Gateway OIDC/API key), models.ts (resolución de modelos).
- rag/: embed.ts (embeddings 1536 dim), search.ts (RPC search\_medical\_knowledge con cliente admin service\_role).
- fda/: client.ts — cliente HTTP openFDA (etiquetas US, límites 429).
- agent/: prompts.ts (system prompt clínico), chat-tools.ts (tools AI: RAG + FDA), graph.ts (LangGraph mínimo / intención).
- persistence/: escritura de sesiones de chat en Supabase.

## 2.3 Integración Supabase (lib/integrations/supabase/)

- client.ts: createBrowserClient (anon / publishable) para el navegador.
- server.ts: createServerClient + cookies() para RSC, Server Actions y rutas que mutan cookies con cuidado.
- admin.ts: createAdminClient (service role) — solo servidor: ingesta RAG, RPC si hiciera falta privilegio.
- middleware.ts: sesión en Edge/middleware.

## 2.4 Validación y tipos

- lib/validation/chat-request.ts: Zod para body del chat (messages, sessionId opcional).
- types/medicoach.ts: tipos compartidos del producto.

## 2.5 Scripts (scripts/)

- ingest-knowledge.ts: Lee data/knowledge/mvp-curated.json (+ opcional openfda-chunks.json), embeddings, insert en medical\_knowledge. Flag --clear para borrar filas mvp-curated / openfda antes de reinsertar.
- fetch-openfda-knowledge.ts: Descarga JSON oficial openFDA (sin scraping HTML), genera data/knowledge/openfda-chunks.json.

## 2.6 Cliente utilitario

- lib/client/speech-recognition.ts: detección Web Speech API y mensajes de error legibles (network, etc.).

---

## 3. Base de datos (Supabase / Postgres)

### 3.1 Extensiones

- uuid-oss, pgvector (vector(1536) para embeddings).

### 3.2 Tablas (public)

| Tabla             | Descripción                                                                                                                                    |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| patient_profiles  | id = auth.users.id, display_name, conditions[], timestamps.                                                                                    |
| medications       | Por paciente: nombre, RxCUI opcional, dosis, frecuencia, schedule jsonb, active.                                                               |
| symptoms          | Por paciente: síntoma, severidad 1–10, nota, medication_id opcional, recorded_at.                                                              |
| chat_sessions     | patient_id, messages jsonb (historial), timestamps.                                                                                            |
| medical_knowledge | RAG: source, category, content, embedding vector(1536), metadata jsonb. Sin políticas SELECT para usuarios finales — ingesta con service_role. |

### 3.3 RLS

- Políticas own row en patient\_profiles, medications, symptoms, chat\_sessions (auth.uid() = patient\_id o id según tabla).
- medical\_knowledge: RLS activado sin políticas públicas → acceso vía service role o RPC controlada.

### 3.4 RPC

- search\_medical\_knowledge(query\_embedding, match\_threshold, match\_count): security definer, similitud coseno, umbral por defecto 0.78.
- Grants: authenticated y (migración adicional) service\_role para ejecutar la función según despliegue.

### 3.5 Auth

- Trigger on\_auth\_user\_created en auth.users → handle\_new\_user() inserta fila en patient\_profiles (on conflict do nothing).

### 3.6 Migraciones

- supabase/migrations/20260423195144\_initial\_schema.sql — esquema base.
- 20260424120000\_grant\_search\_medical\_knowledge\_service\_role.sql — grant RPC a service\_role.

---

## 4. Variables de entorno (referencia)

Ver .env.example: Supabase URL + anon/publishable + SUPABASE\_SERVICE\_ROLE\_KEY; AI Gateway (VERCEL\_OIDC\_TOKEN / AI\_GATEWAY\_API\_KEY) o proveedores directos (ANTHROPIC\_API\_KEY, OPENAI\_API\_KEY, OPENAI\_CHAT\_MODEL); NEXT\_PUBLIC\_SITE\_URL para magic links; openFDA API key opcional.

---

## 5. Integración MCP (orientación)

El producto hoy expone capacidades como: chat con tools (RAG Postgres + openFDA), auth Supabase, persistencia de turnos. Un MCP server podría:

1. Exponer tools alineadas a las ya existentes: búsqueda semántica (misma RPC o vía API interna), consulta FDA (mismo cliente o proxy), lectura/escritura controlada de symptoms / medications si se define contrato y permisos.
2. Reutilizar SUPABASE\_SERVICE\_ROLE\_KEY solo en procesos server-side confiables; para MCP orientado a usuario final, preferir JWT del usuario + cliente con RLS.
3. No duplicar lógica de streaming del chat en MCP salvo que el caso de uso sea otro agente (IDE, otra app); en ese caso MCP actúa como capa de herramientas\*\* y el chat web sigue usando /api/chat.

---

## 6. Dependencias clave (npm)

next, react, react-dom, ai, @ai-sdk/react, @ai-sdk/anthropic, @ai-sdk/openai, @supabase/ssr, @supabase/supabase-js, @langchain/langgraph, @langchain/core, @teispacex/next-themes, sonner, zod, lucide-react, tailwindcss, shadcn (CLI), tsx (scripts).

---

\*Documento generado para documentación interna / handoff. No sustituye prospectos médicos ni asesoramiento clínico.\*